P6.py - create model

```
# import lib
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import classification_report
from pickle import dump
from nltk import word_tokenize
from string import punctuation
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer

# load the data
data = pd.read_csv("news_train.csv")
print(data)

# check and remove null data
print(data.isnull().sum())

# clean the data
sw = stopwords.words("english")
sw.remove("not")
sw.remove("don't")
ps = PorterStemmer()
def clean_text(txt):
	txt = txt.lower()
	txt = word_tokenize(txt)
	txt = [t for t in txt if t not in punctuation ]
	txt = [t for t in txt if t not in sw ]
	txt = [ps.stem(t) for t in txt]
	txt = " ".join(txt)
	return txt

data["clean_text"] = data["text"].apply(clean_text)

# features and Target
tv = TfidfVectorizer()
vector = tv.fit_transform(data["clean_text"])
features = pd.DataFrame(vector.toarray(), columns=tv.get_feature_names_out())
print(features)
target = data["label"]
```

```
# train and test
x_train, x_test, y_train, y_test = train_test_split(features.values, target)

# model
model = MultinomialNB()
model.fit(x_train, y_train)

# cr
y_pred = model.predict(x_test)
cr = classification_report(y_test, y_pred)
print(cr)

# save model and tv
with open("news_model.pkl", "wb") as f:
	dump(model, f)
	print("model saved ")

with open("news_tv.pkl", "wb") as f:
	dump(tv, f)
	print("tv saved")
```

P7.py use the model

```
# import lib
import pandas as pd
from pickle import load
from nltk import word_tokenize
from string import punctuation
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer

# load the data
data = pd.read_csv("news_test.csv")
print(data)

# check and remove null data
print(data.isnull().sum())

# clean the data
sw = stopwords.words("english")
sw.remove("not")
sw.remove("don't")
ps = PorterStemmer()
def clean_text(txt):
	txt = txt.lower()
	txt = word_tokenize(txt)
	txt = [t for t in txt if t not in punctuation ]
	txt = [t for t in txt if t not in sw ]
	txt = [ps.stem(t) for t in txt]
	txt = " ".join(txt)
	return txt

data["clean_text"] = data["text"].apply(clean_text)

# restore model and tv
with open("news_model.pkl", "rb") as f:
	model = load(f)
	print("model back ")

with open("news_tv.pkl", "rb") as f:
	tv = load(f)
	print("tv back")

# prediction
```